PYTHON PROGRAMMING PROJECT REPORT

Documentation for Programs 1–10

> *This report contains the problem statement, objectives, assumptions, algorithm, flowchart representation, pseudocode, test cases, sample console output screens, and conclusion for each supplied Python program. Note: Output screens are sample console-output captures prepared from the supplied logic. Program 9 is interactive.*

1. University Academic Progression Audit

1. Problem Statement

Develop a Python program that audits students' academic progression by checking subject marks, attendance percentage, and prerequisite completion. Eligible only when marks are above minimum, attendance meets requirements, and prerequisites are complete.

2. Objectives

Identify failed subjects, verify 75% minimum attendance, check prerequisites, and calculate averages with eligibility reasons.

3. Assumptions & Algorithm

Assumptions: Pass mark is 40; min attendance 75%; prerequisites provided as boolean.
Algorithm: Start → Set thresholds → Read records → Check failed subjects → Check attendance/prerequisites → Calculate average → Display eligibility → Stop.

4. Test Cases & Output

Test Input / Case	Condition	Expected Result
S101	One subject below 40	NOT ELIGIBLE; DBMS failure
S102	Attendance 70%	NOT ELIGIBLE; attendance below minimum
S103	Prerequisites False	NOT ELIGIBLE; prerequisites incomplete
S104	Marks ≥40, Att 80%, Prereq True	ELIGIBLE

```
      Console Output – Program 1: S101 - Asha | Average: 58.3% | Attendance: 82% | NOT
ELIGIBLE (Failed: DBMS) S102 - Rahul | Average: 54.7% | Attendance: 70% | NOT ELIGIBLE
(Attendance below minimum) S103 - Nina | Average: 81.0% | Attendance: 91% | NOT ELIGIBLE
(Prerequisites incomplete) S104 - Arjun | Average: 43.0% | Attendance: 80% | ELIGIBLE |
Requirements: None
```

Conclusion: Automates academic eligibility checking and clearly reports progression blocks.

2. Advanced String-Based Registration Record Analyzer

1. Problem Statement

Create a registration-record analyzer that splits pipe-separated records, validates field count/formats, and checks duplicate IDs and emails.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
R001	Correct formatting	Valid record
R003	Duplicate details	Duplicate email detected
R004	Bad email, 5-digit phone, age 17	Invalid record with field errors

```
Console Output – Program 2: Record: R001|Anaya Rao|anaya@example.com|9876543210|20 |
Status: VALID | Errors: None Record: R003|Anaya Rao|anaya@example.com|9876543210|20 | Status:
INVALID | Errors: ['Duplicate email'] Record: R004|R004|Li|bad-email|12345|17 | Status:
INVALID | Errors: ['Invalid name', 'Invalid email', 'Invalid phone']
```

3. Smart Campus Incident and Maintenance Prioritization

1. Problem Statement & Objectives

Rank campus maintenance incidents using severity, report frequency, and high-risk location bonuses to label cases as urgent, high, or normal.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
I101	Severity 5, frequency 3, Laboratory	Score 59; URGENT
I102	Severity 2, frequency 8, Hostel	Score 36; NORMAL
I105	Severity 4, frequency 4, Main entrance	Score 51; HIGH

```
Priority order: I101 | Exposed electrical wire | Laboratory | Severity 5 | Reports 3 |
Score 59 | URGENT I104 | Blocked emergency exit | Library | Severity 5 | Reports 2 | Score 57
| URGENT I105 | Damaged pathway | Main entrance | Severity 4 | Reports 4 | Score 51 | HIGH
I103 | Broken classroom fan | Classroom | Severity 3 | Reports 5 | Score 40 | HIGH I102 |
Leaking tap | Hostel | Severity 2 | Reports 8 | Score 36 | NORMAL
```

Conclusion: Provides a consistent rule-based ranking to address safety-critical incidents first.

4. Disaster Relief Supply Allocation

1. Problem Statement & Objectives

Distributes limited water, food, and blankets to areas according to priority, displaying allocated quantities, unmet needs, and remaining stock.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Area North	Priority 1; water need 50, stock 100	Allocate 50; unmet 0
Area South	Priority 3; blankets need 20 after earlier allocations	Allocate 5; unmet 15

```
Area North (Priority 1) | Water: Needed 50, Allocated 50, Unmet 0 | Food: Needed 35, Allocated 35, Unmet 0 Area East (Priority 2) | Water: Needed 45, Allocated 45, Unmet 0 | Food: Needed 40, Allocated 40, Unmet 0 Area South (Priority 3) | Water: Needed 30, Allocated 5, Unmet 25 | Food: Needed 25, Allocated 5, Unmet 20 Remaining stock: {'water': 0, 'food': 0, 'blankets': 0} Total unmet demand: {'water': 25, 'food': 20, 'blankets': 15}
```

Conclusion: Allocates scarce supplies by priority and makes shortages visible for relief planning.

5. E-Commerce Order and Revenue Reconciliation

1. Problem Statement & Objectives

Reconcile e-commerce orders with inventory, product prices, discounts, dispatched quantities, and returns to calculate net revenue and discrepancies.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
O101	Keyboard; 3 ordered/dispatched; 10% discount	Dispatch value ₹2160; no discrepancy
O102	Mouse; 5 dispatched, 1 returned; 5% discount	Dispatch value ₹1900; refund ₹380

```
Order: 0101 | Product: Keyboard | Ordered: 3, Dispatched: 3, Returned: 0 Dispatch value: ₹2160.00 | Refund: ₹0.00 | Net revenue: ₹2160.00 | Status: MATCH Order: 0102 | Product: Mouse | Ordered: 5, Dispatched: 5, Returned: 1 Dispatch value: ₹1900.00 | Refund: ₹380.00 | Net revenue: ₹1520.00 Gross revenue: ₹9460.00 | Refunds: ₹380.00 | Net revenue: ₹9080.00 Remaining inventory: {'Keyboard': 16, 'Mouse': 21, 'Headphones': 13}
```

Conclusion: Combines inventory tracking with revenue reconciliation and highlights order discrepancies.

6. Multi-Criteria Library Resource Distribution

1. Problem Statement & Objectives

Allocates available book copies to branches based on branch priority, reporting remaining inventory and unmet demand.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Central	Python demand 12, stock 30	Allocate 12; unmet 0
South	Python demand 10 after Central/North	Allocate 3; unmet 7

```
Central Branch (Priority 1) | Python: Req 12, Alloc 12, Unmet 0 | Databases: Req 8, Alloc 8, Unmet 0 North Branch (Priority 2) | Python: Req 15, Alloc 15, Unmet 0 | Databases: Req 10, Alloc 10, Unmet 0 South Branch (Priority 3) | Python: Req 10, Alloc 3, Unmet 7 | Databases: Req 6, Alloc 2, Unmet 4 Remaining inventory: {'Python': 0, 'Databases': 0, 'Networking': 0} Unmet demand: {'Python': 7, 'Databases': 4, 'Networking': 4}
```

Conclusion: Supports fair, priority-based distribution of limited library resources.

7. Integrated Manufacturing Production Planner

1. Problem Statement & Objectives

Estimates raw-material requirements, good units after defects, shortages, and defect-rate quality violations for multiple products.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Chair	80 planned, 4 defects, 2 material/unit	Material 160; defect rate 5%; quality OK
Table	45 planned, 6 defects, 5 material/unit	Material 225; defect rate 13.33%; violation

```
      Product: Chair | Material required: 160 | Good units: 76 | Available: 106 | Shortage: 0
| Defect rate: 5.00% | Quality: OK Product: Table | Material required: 225 | Good units: 39 |
Available: 49 | Shortage: 1 | Defect rate: 13.33% | Quality: VIOLATION Total material
required: 565 | Material feasibility: FEASIBLE
```

Conclusion: Identifies material feasibility and quality problems prior to manufacturing.

8. Smart Public Transport Fare Reconciliation

1. Problem Statement & Objectives

Calculates expected revenue from passenger categories, compares with collected revenue, and reports short/excess collections.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Route A adult	20 passengers at ₹30; ₹600 collected	Expected ₹600; reconciled
Route B senior	8 passengers at ₹20; ₹150 collected	Expected ₹160; short collection ₹10

```
      Route A | adult x 20 | Expected: 600 | Collected: 600 | Difference: 0 | Reconciled
Route B | senior x 8 | Expected: 160 | Collected: 150 | Difference: -10 | Short collection
Route C | adult x 15 | Expected: 450 | Collected: 420 | Difference: -30 | Short collection
Total expected: 1480 | Total collected: 1440 | Overall difference: -40
```

Conclusion: Helps transport operators detect revenue discrepancies against expected fares.

9. Smart Campus Management Simulation

1. Problem Statement & Objectives

Menu-driven campus simulation for listing students, recording attendance, adding marks, registering events, and allocating resources.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Choice 1	List students	Displays ST101 Asha and ST102 Ravi
Choice 5	Allocate 2 Projectors	Stock decreases from 5 to 3

```
      SMART CAMPUS MANAGEMENT 1. List students | 2. Record attendance | 3. Add subject mark |
4. Register event | 5. Allocate resources | 6. Summary | 0. Exit Choose option: 1 -> ST101
Asha | Attendance: [] | Marks: [] | Events: [] Choose option: 5 -> Available: {'Projectors':
5, ...} | Allocate: 2 Projectors | Remaining: 3
```

Conclusion: Demonstrates interactive campus operations with validation and summaries.

10. Multi-Event Sports Scheduling and Scoring

1. Problem Statement & Objectives

Displays schedules, detects time overlaps for the same team on the same day, totals team points, and ranks teams.

2. Test Cases & Output

Test Input / Case	Condition	Expected Result
Falcons Monday	Football 10–12 and Basketball 11–13	Conflict detected
Rankings	Falcons 8, Sharks 5, Tigers 4	Falcons first; Sharks second; Tigers third

```
      Schedule: ('Football', 'Falcons', 'Monday', 10, 12, 3), ('Basketball', 'Falcons',
'Monday', 11, 13, 2)... Scheduling conflicts: Falcons : Football overlaps with Basketball
Final rankings: 1. Falcons: 8 points | 2. Sharks: 5 points | 3. Tigers: 4 points
```

Conclusion: Checks team scheduling conflicts and produces deterministic point-based rankings.